# Portada y presentación

*Saludo, título del trabajo y metáfora visual de la semilla.*

### 🗣️ QUÉ DECIR

- Buenas tardes. Soy **Nicolás Rey Alonso** y voy a presentar mi TFG.
- Título: «**Framework backend–frontend para el desarrollo rápido de proyectos apoyado en un modelo de datos en evolución**».
- Aplicación a los proyectos con el **Jardín Botánico Viera y Clavijo**. Lo he llamado **UniWorks**.
- Tutores: **Rafael Juan Nebot Medina** (ITC) y **María Dolores Afonso Suárez** (Escuela).
- La imagen de la portada resume la idea: una **única semilla** —el modelo de datos— de la que crece todo lo demás.

### 🔑 CLAVES & ACLARACIONES

- Framework = dos artefactos: `uniback` (backend Python) + `ngt-gui` (librería Angular).
- Hablar despacio aquí: marca el tono de toda la defensa.
- Mantener contacto visual con el tribunal al presentarse.

→ **Empiezo por el problema que motiva todo.**

# El problema

***Cuando el modelo cambia, todo lo demás se reescribe a mano.***

### 🗣️ QUÉ DECIR

- El ITC desarrolla, con el Jardín Botánico, aplicaciones de gestión de datos botánicos. Su **modelo de datos no es estable**: crece con la investigación.
- Cada cambio se **propaga en cascada**: backend (modelos, validación, endpoints), comunicación (contratos, tipos) y frontend (formularios, tablas, vistas).
- Casi todo ese código se **escribe a mano** y queda **acoplado a un dominio**.
- El resultado es **caro de mantener** y **difícil de reutilizar**.

### 🔑 CLAVES & ACLARACIONES

- Señalar el diagrama de cascada en la diapositiva: Modelo → Backend → Comunicación → Frontend, todo «a mano».
- Tres capas afectadas = tres oportunidades de error y repetición.

→ ¿Cuál es la propuesta para resolverlo?

# La propuesta

***El modelo de datos como única fuente de verdad.***

### 🗣️ QUÉ DECIR

- El desarrollador escribe **solo el modelo ORM** en SQLAlchemy.
- El framework **deriva** un JSON Schema enriquecido con pistas de presentación.
- Ese esquema viaja por un **contrato de comunicación uniforme**.
- En el otro extremo se **genera el formulario**, sin escribir HTML específico de cada entidad.
- Son dos piezas: **uniback** (FastAPI + PostgreSQL) y **ngt-gui** (Angular). Un cambio en el modelo **se propaga solo**.

### 🔑 CLAVES & ACLARACIONES

- Cadena: Modelo ORM → Esquema enriquecido ( `JSON Schema + x-*` ) → Contrato uniforme ( `content · count · issues` ) → Formulario ( `<ngt-dynamic-form>` ).
- Recalcar que el front **nunca necesita conocer** la forma concreta de una entidad.

→ ¿Qué había antes de empezar?

# Punto de partida

*Qué recibí y qué aporta este trabajo.*

🗣️ QUÉ DECIR

- **Fase 0** (inicio prácticas): app Angular monolítica de NextGenDem, con **178 métodos** escritos a mano. Nada reutilizable.
- En prácticas extraje sus componentes a **ngt-gui**.
- **Fase 1** (inicio TFG): ngt-gui (105 componentes, atada a un dominio) + uniback del ITC (≈ 20 entidades con CRUD). Sin descubrimiento, sin pistas, sin plugins. **Dos piezas que no encajan.**
- **Mi aportación** es todo lo que separa la Fase 1 de la **Fase 2**: el framework UniWorks con 61 entidades, descubrimiento, cliente genérico, formularios del esquema y dominio en plugins.

🔑 CLAVES & ACLARACIONES

- Delimitar bien: el núcleo ORM y un primer CRUD venían del ITC. Yo añadí el contrato, el enriquecimiento, el descubrimiento y la cadena completa.
- Si preguntan: la librería NGT sale de las prácticas; su generalización es el TFG.

→ ¿En qué objetivos concretos se desglosa?

# Cinco objetivos específicos

***Un framework reutilizable, en cinco objetivos.***

🗣 QUÉ DECIR

- **OE1 — API modular**: envoltorio en toda la API, catálogo de errores y fábricas CRUD.
- **OE2 — Modelo de datos**: núcleo ORM consolidado, con versionado e histórico.
- **OE3 — Comunicación estandarizada**: descubrimiento de entidades y cliente genérico tipado.
- **OE4 — Formularios automáticos**: generados con Formly a partir del esquema.
- **OE5 — Esquemas desde el ORM**: el modelo como única fuente de verdad.

🔑 CLAVES & ACLARACIONES

- Requisitos priorizados con MoSCoW: 10 funcionales y 7 no funcionales (reutilización, portabilidad, compatibilidad con heredado).
- Los cinco quedan **COMPLETADOS** al cierre y **respaldados por pruebas**.
- Cada OE se mapea con un incremento del desarrollo.

→ ¿Qué existe ya fuera? Estado del arte.

# Estado del arte

***Los renderizadores ya existen. Faltaba la cadena completa.***

### 🗣️ QUÉ DECIR

- **react-jsonschema-form** y **JSON Forms**: lado cliente, asumen un esquema ya dado.
- **OpenAPI / Swagger**: describe operaciones, no presentación.
- **Admin autogenerados** (Django Admin): parten del ORM, pero acoplan la interfaz al servidor.
- **Aportación de UniWorks**: completar la cadena. Derivar el esquema desde el ORM, enriquecerlo con pistas, transportarlo por un contrato uniforme y versionado, y consumirlo con una capa genérica.

### 🔑 CLAVES & ACLARACIONES

- Tabla: columnas Parte del ORM · Pistas UI · Contrato · Versionado.
- Solo UniWorks cubre las cuatro: Sí · Sí (x-*) · Uniforme · ETag.
- Frase fuerte: la aportación **no es el renderizador**, es **cerrar la cadena**.

→ ¿Cómo encaja todo? Arquitectura.

# Arquitectura general

***Dos sistemas y un contrato como única superficie de acoplamiento.***

🗣️ QUÉ DECIR

- **Izquierda — uniback**: modelo SQLAlchemy → Pydantic con pistas `x-*` → JSON Schema enriquecido.
- **Centro — Contrato HTTP**: toda respuesta viaja en `{ content, count, issues }`. Tres endpoints clave: esquema individual, bundle y descubrimiento.
- **Derecha — ngt-gui**: `EntityClient` pide el esquema, `schemaToFormly()` lo traduce, `<ngt-dynamic-form>` lo renderiza.
- El frontend **nunca necesita conocer** la forma concreta de una entidad.

🔑 CLAVES & ACLARACIONES

- Endpoints: `GET /<e>/schema.json` · `GET /api/sys/schemas` [ETag] · `GET /api/sys/entities`.
- Envoltorio: `{ content, count, issues }` — éxito y error, mismo formato.
- Esta es la diapositiva-resumen: si me pierdo, vuelvo a este flujo.

→ El corazón del modelo de datos.

# Núcleo polimórfico

***Un núcleo que trata igual lo que es distinto.***

### 🗣️ QUÉ DECIR

- Toda entidad hereda de `FunctionalObject`, asociada a un `ObjectType` con código numérico.
- Anotaciones, permisos (ACL) y colecciones apuntan a «cualquier objeto funcional» **sin conocer su clase concreta**.
- La autorización usa el mismo patrón: `Identity` y `Role` heredan de `Authorizable`.
- Consecuencia práctica: **Plant** (un plugin) hereda todo —anotaciones, permisos, colecciones— sin que el núcleo sepa que existe.
- Modelo ampliado del ITC, al que añadí **versionado con SQLAlchemy-Continuum**.

### 🔑 CLAVES & ACLARACIONES

- Señalar el diagrama: ObjectType → FunctionalObject → Collection / Screen / Plant (plugin, borde discontinuo).
- Cuando la clase es raíz de jerarquía, el esquema emite `oneOf` + `x-discriminator`.
- Conecta con el Incremento 4 (bundle).

→ Cómo entra el dominio sin tocar el núcleo: plugins.

# Plugins y micronúcleo

*El dominio entra desde fuera: plugins sobre un micronúcleo.*

### 🗣️ QUÉ DECIR

- Un **plugin** hereda de `UnibackPlugin` y sobreescribe solo los hooks que necesita.
- Ejemplo — **plugin de plantas**: declara sus modelos, expone su CRUD con una fábrica y aporta un widget de mapa. Nada más.
- Refactoricé el backend hacia un **micronúcleo**: cada dominio (auth, ficheros, anotaciones, especies) es un plugin de serie.
- Despliegue **multinodo**: misma imagen Docker, `UNIBACK_NODE_TYPE` decide qué plugins sirve, **Traefik** enruta por prefijo. Un nodo nuevo se **añade en caliente**.

### 🔑 CLAVES & ACLARACIONES

- Hooks: `get_model_modules`, `get_routers`, `get_field_widgets`.
- Materializa el requisito de **reutilización**: el núcleo no acopla lógica de dominio.
- Diagrama derecho: nodos core · auth · species · files · annotations · assistant + seedbeds (en caliente).

→ ¿Cómo se desarrolló? Metodología.

# Metodología

***Iterativo incremental: cuatro incrementos.***

### 🗣 QUÉ DECIR

- Metodología **iterativa incremental** acordada con la empresa. Cada incremento entrega una versión funcional.
- Encaja con el problema: la propia metodología absorbe la incertidumbre de un modelo que evoluciona.
- **Cuatro incrementos**: 1) API y modelado · 2) Contrato · 3) Formularios · 4) Esquemas.
- Horas: estimé **265**, fueron **290**. La desviación está en los tres primeros (curva de aprendizaje del código heredado); el cuarto se cerró por debajo.

### 🔑 CLAVES & ACLARACIONES

- Señalar las barras estimado vs. real en la diapositiva.
- Descarté Scrum: línea de trabajo independiente, sin equipo.
- Incremento 1: entorno Docker + núcleo versionado (48 h).
- Total: Planificación 48 + Inc1 56 + Inc2 62 + Inc3 58 + Inc4 34 + Doc 32 = 290 h.

→ **Primer incremento de desarrollo: el contrato.**

# Incremento 2 · Contrato

***Una sola forma para cualquier respuesta.***

### 🗣️ QUÉ DECIR

- El contrato **es universal**: toda respuesta tiene la misma forma — `content`, `count`, `issues`.
- **También los errores**: manejadores globales + catálogo de siete códigos estándar. El frontend nunca recibe dos formas distintas de fallo.
- **Fábricas CRUD** generalizadas: seis endpoints por entidad, de ≈ 20 a **61 entidades**.
- **Descubrimiento**: el frontend pregunta qué entidades existen; no las lleva escritas.
- Capa **aditiva**: el servicio heredado sigue funcionando sin tocarlo.

### 🔑 CLAVES & ACLARACIONES

- Señalar los dos bloques de código: respuesta de éxito (borde verde) vs. error de validación (borde naranja).
- Convención: `GET` lista · `GET /{id}` · `POST` · `PUT` · `DELETE` · `GET /schema.json`.
- Si preguntan: `VALIDATION_ERROR` lleva `location` (viene de Pydantic).

**→ La principal aportación: del esquema al formulario.**

# Incremento 3 · Formularios

***Del esquema al formulario, sin escribir HTML.***

### 🗣️ QUÉ DECIR

- **Backend**: amplié la conversión SQLAlchemy → Pydantic para inyectar una **pista de presentación por columna**: Boolean→checkbox, DateTime→datepicker, Enum→select, String largo→textarea, FK→select con carga diferida.
- Son valores por defecto: lo que el desarrollador **declare a mano siempre gana**. Viajan dentro del esquema → sobreviven al cacheo.
- **Frontend**: `schemaToFormly` es un conversor puro que traduce el esquema a Formly, con validadores y condiciones de visibilidad.
- El resultado: **una etiqueta y la ruta**. El componente descubre, pide, monta, valida y envía.

### 🔑 CLAVES & ACLARACIONES

- Señalar la tabla izquierda (columna → pista) y el código derecho: `<ngt-dynamic-form entityPath="/roles">`.
- «La heurística propone; el desarrollador siempre puede sobrescribirla.»
- Esta es la diapositiva que demuestra la tesis central.

# Incremento 4 · Esquemas

***Todos los esquemas, en un bundle versionado.***

### 🗣️ QUÉ DECIR

- Cada fábrica CRUD registra su entidad en un **registro global**; se construye un **bundle**: un único documento con todos los esquemas enriquecidos.
- Su **versión = ETag**: si nada ha cambiado, el servidor responde **304**.
- **Polimorfismo**: las subclases viajan como `oneOf` con su discriminador.
- **Relaciones del ORM** reflejadas en `x-relationships`.
- Todo es **determinista**: mismo modelo, misma versión.

### 🔑 CLAVES & ACLARACIONES

- Señalar el bloque de código con el JSON del bundle: `version`, `entities`, `oneOf`, `x-discriminator`, `x-relationships`.
- Valor extra: un script externo puede pedir el bundle y generar tipos TypeScript, validadores o documentación.
- Decisión: variantes incrustadas (no `$ref`) → bundle autocontenido.

**→ Paso a las demostraciones.**

# Demo 1 · Uso general

***Autenticación, navegación y CRUD generado desde el esquema.***

### 🗣️ QUÉ DECIR

- Recorre el **uso general** de la aplicación de demostración.
- Autenticación, **navegación por entidades descubiertas** y CRUD generado.
- La lista, el alta y la edición salen del **esquema de cada entidad**: no hay ninguna pantalla escrita a mano por entidad.
- Duración: **≈ 55 segundos**.

### 🔑 CLAVES & ACLARACIONES

- Presentar lo que van a ver **antes** de reproducir el vídeo.
- Tecla P pausa/reanuda el vídeo.
- Si preguntan: la navegación lateral la genera el descubrimiento de `/api/sys/entities`.

→ **Demo 2: extensibilidad con plugins.**

# Demo 2 · Un plugin en caliente

***Se levanta un nodo nuevo y sus entidades aparecen sin reiniciar.***

### 🗣️ QUÉ DECIR

- Muestra la **extensibilidad**: con el sistema en marcha, levanto un **nodo nuevo** —semilleros— que es un plugin independiente.
- **Traefik lo detecta**, el descubrimiento lo incorpora y sus entidades aparecen en la interfaz **sin reiniciar nada** ni tocar el frontend.
- Duración: **≈ 34 segundos**.

### 🔑 CLAVES & ACLARACIONES

- Presentar brevemente: «Voy a levantar un nodo nuevo con Docker Compose y veremos cómo aparece solo.»
- Demuestra que la arquitectura multinodo funciona en la práctica.
- Conecta con la diapositiva 09 (Plugins y micronúcleo).

→ Demo 3: pestañas y asistente de IA.

# Demo 3 · Pestañas y asistente de IA

***El asistente lee el esquema y propone altas; el usuario confirma.***

### 🗣️ QUÉ DECIR

- Dos herramientas añadidas sobre el framework: el **sistema de ventanas y pestañas** y el **asistente de IA** (un plugin más).
- El asistente funciona con modelos locales (LM Studio) o en la nube. **Consulta el esquema** de la entidad y **propone altas bien formadas**.
- Patrón **proponer y confirmar**: nada se escribe hasta que el usuario acepta. Siempre bajo los permisos de su sesión.
- Duración: **≈ 1 minuto 13 segundos**.

### 🔑 CLAVES & ACLARACIONES

- Presentar lo que van a ver antes de reproducir.
- Recalcar: el asistente usa la misma infraestructura (esquema, EntityClient) que el resto del framework.
- Si preguntan por la IA: es un valor añadido sobre el framework, no el objeto central del TFG.

→ **Resultados cuantitativos.**

# Resultados en cifras

***Una entidad nueva: ≈ 35 líneas de backend y ninguna de frontend.***

🗣️ QUÉ DECIR

- La cifra clave: exponer una entidad nueva cuesta **≈ 35 líneas de backend** y **cero de frontend**.
- **61 entidades** expuestas por la cadena ORM–esquema, en nueve dominios.
- **366 endpoints CRUD** generados (vs. 178 métodos a mano del proyecto original).
- **39 extensiones** `x-*` reconocidas en el esquema.
- **41 tests backend** (45 % cobertura) + **17 tests unitarios** del conversor esquema–Formly.

🔑 CLAVES & ACLARACIONES

- Señalar las cinco cifras grandes en la diapositiva.
- Contraste: 366 generados vs. 178 a mano (catálogo mucho menor).
- Si preguntan por cobertura: estrategia de tests en piezas puras + integración de la API.

→ **El resultado más significativo: la comparación cualitativa.**

# Antes y después

***El resultado más significativo es cualitativo.***

🗣️ QUÉ DECIR

- **Antes**: router a mano, esquemas Pydantic a mano, errores ad hoc, un método por operación en el frontend, HTML campo a campo.
- **Después**: una fábrica CRUD, validación derivada del ORM, envoltorio uniforme, `EntityClient.for(path)`, `<ngt-dynamic-form>`.
- La última fila responde al problema: un cambio en el modelo ya no se propaga a mano a tres capas, **se propaga solo**.
- **El esfuerzo deja de crecer** con el número de entidades y con la frecuencia de cambios.

🔑 CLAVES & ACLARACIONES

- Señalar la tabla fila por fila: Endpoints · Validación · Errores · Cliente · Formulario · **Cambio en el modelo** (fila destacada).
- Esta tabla cierra el arco narrativo que empezó en la diapositiva 02.

→ Conclusiones y líneas futuras.

# Conclusiones y trabajo futuro

***El coste de un modelo que cambia deja de crecer.***

🗣️ QUÉ DECIR

- Los **cinco objetivos**, completados y respaldados por pruebas.
- Un contrato y una cadena de generación que **desacoplan el framework del dominio**.
- Validado sobre **necesidades reales del ITC** con el Jardín Botánico.
- **Trabajo futuro**: 1) Codegen desde el bundle. 2) Migrar el servicio heredado al cliente genérico. 3) `oneOf` en el conversor Formly. 4) Soporte de criptografía.
- Sobre la **IA**: la usé como apoyo para explorar el código heredado y depurar, siempre bajo mi supervisión; las decisiones de diseño han sido mías.

🔑 CLAVES & ACLARACIONES

- ODS: relación alta con ODS 9 (industria e innovación), media con 15 y 8.
- Si preguntan por limitaciones: el conversor aún no soporta `oneOf` y la cobertura de tests puede crecer.
- Tono: cerrar con seguridad, sin prisa.

→ **Agradecimientos y turno de preguntas.**

# Gracias

***Quedo a disposición del tribunal.***

🗣️ QUÉ DECIR

- Muchas gracias por su atención.
- Quiero agradecer a **Rafael Nebot** y al ITC la oportunidad de participar en este proyecto, y a **Marilola Afonso** su apoyo durante el desarrollo.
- El código está disponible en **github.com/NicolasReyAlonso/UniWorks**.
- Quedo a disposición del tribunal para las preguntas que consideren.

🔑 CLAVES & ACLARACIONES

- Sonreír. Mantener contacto visual.
- No añadir ideas nuevas aquí: el cierre es limpio y breve.
- Tener preparado un vaso de agua para las preguntas.
- Lema: «El modelo ORM como única fuente de verdad: el resto se deriva.»

**Fin — turno de preguntas.**